

19 – Pagination, images upload & User Management

ONLY for grades 4 and 5 (look at the **requirements** for more details)

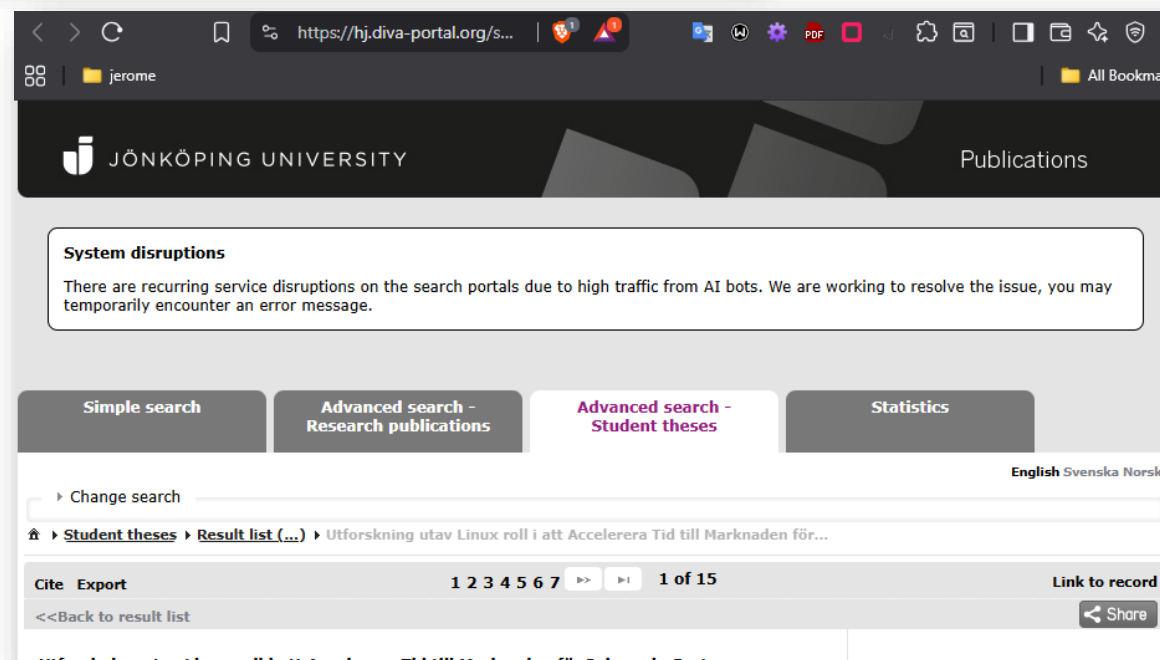
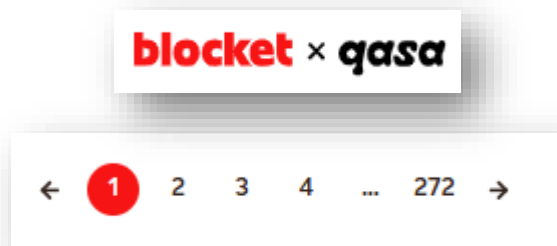
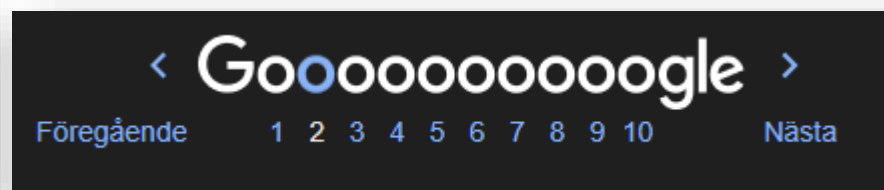
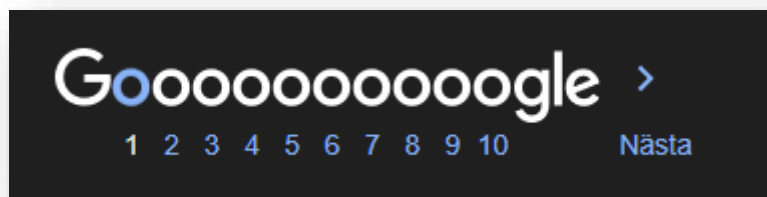
Jérôme Landré – jerome.landre@ju.se

OUTLINE

- 1) Pagination
- 2) Pagination in your project
- 3) Images upload: multer
- 4) User Management System

1) PAGINATION

- A pagination system helps you display a list of pages to select on a website:



1) PAGINATION

- A pagination system helps you display a subset of a SQL SELECT query using LIMIT and OFFSET
 - LIMIT defines the number of elements to display
 - OFFSET defines the starting offset in the list of all elements

DB Browser for SQLite - C:\Users\lanjer\Desktop\lecture-pages-multer\portfolio-jl-03-sessions\my-project-data.sqlite3.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Undo Open Project Save Project Attach Database Close Database

Database Structure Browse Data Edit Pragmas Execute SQL

Table: projects

pid	pname	pyear	pdesc	ptype	pimgURL
Filter	Filter	Filter	Filter	Filter	Filter
1	Counting people with a camera	2022	The purpose of this project is to ...	Research	/img/counting.png
2	Visualisation 2 of 3D medical images	2020	The project makes a 3D model of the...	Research	/img/medical.png
3	Multiple questions system	2021	During the lockdowns in France, thi...	Teaching	/img/qcm07.png
4	Image comparison with the Local ...	2020	The project is about finding and ...	Research	/img/diaw02.png
5	Management system for students' ...	2012	This project was about the creation...	Teaching	/img/management.png
6	Magnetic Resonance Spectroscopy	2013	Analysis of signals and images from...	Research	/img/yu00.png
7	Signal 2 Analysis for Detection of ...	2020	This project 3 was about the ...	Teaching	/img/yu00.png

DB Browser for SQLite - C:\Users\lanjer\Desktop\lecture-pages-multer\portfolio-jl-03-sessions\my-project-data.sqlite3.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Undo Open Project Save Project Attach Database Close Database

Database Structure Browse Data Edit Pragmas Execute SQL

SQL 1*

```
1 SELECT * FROM projects LIMIT 3 OFFSET 0;
```

pid	pname	pyear	pdesc	ptype	pimgURL
1	Counting people with a camera	2022	The purpose of this project is to ...	Research	/img/counting.png
2	Visualisation 2 of 3D medical images	2020	The project makes a 3D model of the...	Research	/img/medical.png
3	Multiple questions system	2021	During the lockdowns in France, thi...	Teaching	/img/qcm07.png

Execution finished without errors.
Result: 3 rows returned in 6ms
At line 1:
SELECT * FROM projects LIMIT 3 OFFSET 0;

DB Browser for SQLite - C:\Users\lanjer\Desktop\lecture-pages-multier\portfolio-jl-03-sessions\my-project-data.sqlite3.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Undo Open Project Save Project Attach Database Close Database

Database Structure Browse Data Edit Pragma Execute SQL

Table: projects

pid	pname	pyear	pdsc	ptype	pimgURL
Filter	Filter	Filter	Filter	Filter	Filter
1	1 Counting people with a camera	2022	The purpose of this project is to ...	Research	/img/counting.png
2	2 Visualisation 2 of 3D medical images	2020	The project makes a 3D model of the...	Research	/img/medical.png
3	3 Multiple questions system	2021	During the lockdowns in France, thi...	Teaching	/img/qcm07.png
4	4 Image comparison with the Local ...	2020	The project is about finding and ...	Research	/img/diaw02.png
5	5 Management system for students' ...	2012	This project was about the creation...	Teaching	/img/management.png
6	6 Magnetic Resonance Spectroscopy	2013	Analysis of signals and images from...	Research	/img/yu00.png
7	7 Signal 2 Analysis for Detection of ...	2020	This project 3 was about the ...	Teaching	/img/yu00.png

DB Browser for SQLite - C:\Users\lanjer\Desktop\lecture-pages-multier\portfolio-jl-03-sessions\my-project-data.sqlite3.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Undo Open Project Save Project Attach Database Close Database

Database Structure Browse Data Edit Pragma Execute SQL

SQL 1*

```
1 SELECT * FROM projects LIMIT 3 OFFSET 2;
```

pid	pname	pyear	pdsc	ptype	pimgURL
1	3 Multiple questions system	2021	During the lockdowns in France, thi...	Teaching	/img/qcm07.png
2	4 Image comparison with the Local ...	2020	The project is about finding and ...	Research	/img/diaw02.png
3	5 Management system for students' ...	2012	This project was about the creation...	Teaching	/img/management.png

Execution finished without errors.
Result: 3 rows returned in 11ms
At line 1:
SELECT * FROM projects LIMIT 3 OFFSET 2;

DB Browser for SQLite - C:\Users\lanjer\Desktop\lecture-pages-multer\portfolio-jl-03-sessions\my-project-data.sqlite3.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Undo Open Project Save Project Attach Database Close Database

Database Structure Browse Data Edit Pragma Execute SQL

Table: projects

pid	pname	pyear	pdesc	ptype	pimgURL
Filter	Filter	Filter	Filter	Filter	Filter
1	1 Counting people with a camera	2022	The purpose of this project is to ...	Research	/img/counting.png
2	2 Visualisation 2 of 3D medical images	2020	The project makes a 3D model of the...	Research	/img/medical.png
3	3 Multiple questions system	2021	During the lockdowns in France, thi...	Teaching	/img/qcm07.png
4	4 Image comparison with the Local ...	2020	The project is about finding and ...	Research	/img/diaw02.png
5	5 Management system for students' ...	2012	This project was about the creation...	Teaching	/img/management.png
6	6 Magnetic Resonance Spectroscopy	2013	Analysis of signals and images from...	Research	/img/yu00.png
7	7 Signal 2 Analysis for Detection of ...	2020	This project 3 was about the ...	Teaching	/img/yu00.png

DB Browser for SQLite - C:\Users\lanjer\Desktop\lecture-pages-multer\portfolio-jl-03-sessions\my-project-data.sqlite3.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Undo Open Project Save Project Attach Database Close Database

Database Structure Browse Data Edit Pragma Execute SQL

SQL 1*

```
1 SELECT * FROM projects LIMIT 4 OFFSET 3;
```

pid	pname	pyear	pdesc	ptype	pimgURL
1	4 Image comparison with the Local ...	2020	The project is about finding and ...	Research	/img/diaw02.png
2	5 Management system for students' ...	2012	This project was about the creation...	Teaching	/img/management.png
3	6 Magnetic Resonance Spectroscopy	2013	Analysis of signals and images from...	Research	/img/yu00.png
4	7 Signal 2 Analysis for Detection of ...	2020	This project 3 was about the ...	Teaching	/img/yu00.png

Execution finished without errors.
Result: 4 rows returned in 6ms
At line 1:
SELECT * FROM projects LIMIT 4 OFFSET 3;

2) PAGINATION IN YOUR PROJECT

- For grade 4 (project alone or by pair of two students):
 - The code for pagination seems complicated **BUT** it is ALWAYS the same:
 - You define the number of elements per page, **numberPerPage**
 - You need to know on which page you are, **currentPage**
 - You need to know how many elements (projects in my case) you have in **total**
 - Then, you compute the number of pages, **pages=ceil(total/numberPerPage)**
 - You create a link First to the first page, **page=1**
 - You create a link Previous to the previous page, **page=currentPage-1**
 - You print all the available **pages** with **links**
 - You create a link Next to the next page, **page=currentPage+1**
 - You create a link Last to the last page, **page=total**


```
app.get('/projects', (request,response) => {  
  // how many projects in total?  
  db.get('SELECT COUNT(*) as totalProjects FROM projects', (err, row) => {  
    if (err) {  
      return res.status(500).send('Database error');  
    } else {  
      const totalProjects = row.totalProjects  
      console.log(`---> Total number of projects in the database: ${totalProjects}`);  
      // how many pages do we need?  
      const totalPages = Math.ceil(totalProjects / numberOfElementsPerPage)  
      console.log(`---> Total number of pages: ${totalPages}`);  
      // which page are we on?  
      let currentPage = parseInt(request.query.page) || 1  
      if (currentPage > totalPages) currentPage = totalPages  
      if (currentPage < 1) currentPage = 1  
      pages=[]  
      for (let i = 1; i <= totalPages; i++) {  
        pages.push(i);  
      }  
    }  
  })  
})
```

```

console.log(`---> Current page: ${currentPage}`);
// calculate the OFFSET for the SQL LIMIT clause
const offset = (currentPage - 1) * numberOfElementsPerPage
console.log(`---> OFFSET for SQL LIMIT clause: ${offset}`);
// get the projects for this page
db.all('SELECT * FROM projects LIMIT ? OFFSET ?', [numberOfElementsPerPage, offset], (err, theProjects) => {
  if (err) {
    console.error(err.message);
    const model = { error: "Error retrieving projects from the database." }
    response.render('projects', model);
  } else {
    console.log(`---> Retrieved ${theProjects.length} projects from the database.`);
    console.log(`---> Projects: ${JSON.stringify(theProjects)}`);
    const model = {
      projects: theProjects,
      currentPage: currentPage,
      prevPage: currentPage - 1,
      nextPage: currentPage + 1,
      totalPages: totalPages,
      hasPrevPage: currentPage > 1,
      hasNextPage: currentPage < totalPages,
      pages: pages
    };
    response.render('projects', model); // the list of projects
  }
})
}
})

```

```
<div>
  <h1>List of Projects</h1>
  <div>
    {{#if session.isAdmin}}
      <p>
        <form style="display:inline" method="GET" action="/projects/new">
          <button type="submit" class="important">New project</button>
        </form>
      </p>
    {{/if}}
    <ul>
      {{#each projects}}
        <li>
          <a href="/projects/{{pid}}">{{pname}}</a>: {{pdesc}} Year: {{pyear}}.
          {{#if ../session.isAdmin}}
            <form style="display:inline" method="GET" action="/projects/modify/{{pid}}">
              <button type="submit" class="important">Edit</button>
            </form>
            <form style="display:inline" method="POST" action="/projects/delete/{{pid}}">
              <button type="submit" class="important">X</button>
            </form>
          {{/if}}
        </li>
      {{/each}}
    </ul>
  </div>
</div>
```

```

<p>
  <a href="/projects?page=1">First</a>
  {{#if hasPrevPage}}
    <a href="/projects?page={{prevPage}}">Previous</a>
  {{/if}}
  {{#each pages}}
    <a href="/projects?page={{this}}">{{this}}</a>
  {{/each}}
  {{#if hasNextPage}}
    <a href="/projects?page={{nextPage}}">Next</a>
  {{/if}}
  <a href="/projects?page={{totalPages}}">Last</a>
</p>
{{#if error}}
  <p class="error">{{error}}</p>
{{/if}}
</div>
</div>

```

2) PAGINATION IN YOUR PROJECT

[Home](#) [Projects](#) [Skills](#) [Contact](#) [Login](#)

List of Projects

- [Image comparison with the Local Dissimilarity Map](#): The project is about finding and quantifying the differences between two images of the same size. The applications were numerous: satellite imaging, medical imaging,... Year: 2020.
- [Management system for students' internships](#): This project was about the creation of a database to manage the students' internships. Year: 2012.
- [Magnetic Resonance Spectroscopy](#): Analysis of signals and images from Magnetic Resonance Spectroscopy and Imaging. Year: 2013.

[First](#) [Previous](#) [1](#) [2](#) [3](#) [Next](#) [Last](#)

[Home](#) [Projects](#) [Skills](#) [Contact](#) Hej, admin! [Logout](#)

List of Projects

[New project](#)

- [Counting people with a camera](#): The purpose of this project is to count people passing through a corridor and to know how many are in the room at a certain time. Year: 2022. [Edit](#) [X](#)
- [Visualisation 2 of 3D medical images](#): The project makes a 3D model of the analysis of the body of a person and displays the detected health problems. It is useful for doctors to view in 3D their patients and the evolution of a disease. Year: 2020. [Edit](#) [X](#)
- [Multiple questions system](#): During the lockdowns in France, this project was useful to test the students online with a Quizz system. Year: 2021. [Edit](#) [X](#)

[First](#) [1](#) [2](#) [3](#) [Next](#) [Last](#)

3) IMAGES UPLOAD: MULTER

Multer

npm

v2.0.2

downloads

33.3M/month

ci

passing

coverage

98%

openssf scorecard

6.9

Multer is a node.js middleware for handling `multipart/form-data`, which is primarily used for uploading files. It is written on top of [busboy](#) for maximum efficiency.

NOTE: Multer will not process any form which is not multipart (`multipart/form-data`).

```
npm install multer
```

```
<form action="/profile" method="post" enctype="multipart/form-data">
  <input type="file" name="avatar" />
</form>
```

```
const express = require('express')
const multer  = require('multer')
const upload = multer({ dest: 'uploads/' })

const app = express()

app.post('/profile', upload.single('avatar'), function (req, res, next) {
  // req.file is the `avatar` file
  // req.body will hold the text fields, if there were any
})
```

3) IMAGES UPLOAD: MULTER

- For grade 5 (project by pair of two students), image upload and CRUD for this image:
 - Firstly, we need to install multer: `npm i multer`
 - Secondly, we must load the package and specify the destination path (our `public/img/` folder)

```
//--- LOAD THE PACKAGES
const express=require('express')
const {engine}=require('express-handlebars')
const bodyParser=require('body-parser')
const bcrypt=require('bcrypt')
const session=require('express-session')
const sqlite3=require('sqlite3') // load the sqlite3 package
const connectSqlite3=require('connect-sqlite3') // store the sessions in a SQLite3 database file
const multer=require('multer') // upload images
const upload=multer({ dest: 'public/img/' })
```

3) IMAGES UPLOAD: MULTR

- Then, we need to change our creation and update forms as well as the code to process them.

```
{{#if p.pid}}  
  <form action="/projects/modify/{{p.pid}}" method="post" id="projectform" enctype="multipart/form-data">  
{{else}}  
  <form action="/projects/new" method="post" id="projectform" enctype="multipart/form-data">  
{{/if}}
```


3) IMAGES UPLOAD: MULTR

```

<div id="main">
  {{#if p.pid}}
    <h1>Modify project</h1>
  {{else}}
    <h1>New project</h1>
  {{/if}}
  <div>
    {{#if p.pid}}
      <form action="/projects/modify/{{p.pid}}" method="post" id="projectform" enctype="multipart/
      form-data">
    {{else}}
      <form action="/projects/new" method="post" id="projectform" enctype="multipart/form-data">
    {{/if}}
    <fieldset>
      <legend>Project</legend>
      <label for="name">Name:</label>
      <input type="text" name="name" id="name"
        {{#if p.pname}} value="{{p.pname}}" {{/if}}
      />
      <br />
      <label for="desc">Description:</label>
      <textarea name="desc" id="desc" rows="4" cols="40">{{#if p.pdesc}}{{p.pdesc}}{{/if}}</
      textarea>
      <br />
      <label for="year">Year:</label>
      <input type="number" name="year" id="year"
        {{#if p.pyear}} value="{{p.pyear}}" {{/if}}
      />
    </fieldset>
  </div>

```

3) IMAGES UPLOAD: MULTR

```

<br />
<label for="type">Type:</label>
<select name="type" id="type">
  <option value="Teaching"
    {{#if (eq p.ptype "Teaching")}}selected{{/if}}
  >Teaching</option>
  <option value="Research"
    {{#if (eq p.ptype "Research")}}selected{{/if}}
  >Research</option>
</select>
<br />
<label for="image">Image:</label>
<input type="file" name="image" id="image" required="required" />
<br />
{{#if p.pid}}
  <input type="submit" value="Modify" />
{{else}}
  <input type="submit" value="Create" />
{{/if}}
</fieldset>
</form>
{{#if error}} <p class="error">{{error}}</p> {{/if}}
</div>
</div>

```

3) IMAGES UPLOAD: MULTR

```
app.post('/projects/new', upload.single('image'), (request, response) => {  
  // the treatment of the data received from the client form  
  console.log(`Here comes the data received from the form on the client: ${JSON.stringify(request.body)}`)  
  const { name, year, desc, type } = request.body  
  const routeImage = '/img/'+request.file.filename  
  if (request.session.isAdmin) {  
    db.run('INSERT INTO projects (pname, pyear, pdesc, ptype, pimgURL) VALUES (?, ?, ?, ?, ?)', [name,  
    year, desc, type, routeImage], (err) => {  
      if (err) {  
        console.error(err.message);  
        const model = { error: "Error inserting the new project into the database." }  
        response.render('home', model);  
      } else {  
        const sqlGetLastId = `SELECT last_insert_rowid() as lastId`;  
        db.get(sqlGetLastId, [], (err, row) => {  
          if (err) {  
            console.error(err.message);  
            return res.status(500).send("Error getting last inserted ID");  
          } else {
```

If you use INNER JOIN in your project, you **MUST** associate a skill to a project when you create the project!

3) IMAGES UPLOAD: MULTER

If you use INNER JOIN in your project, you **MUST** associate a skill to a project when you create the project!



```

db.run('INSERT INTO projectshaveskills (pid, sid) VALUES (?, ?)', [row.lastId, 1],
(err) => {
  if (err) {
    console.error(err.message);
    const model = { error: "Error inserting the new project into the database." }
    response.render('home', model);
  } else {
    console.log('New project created!')
    response.redirect('/')
  }
});
});
}
});
} else {
  model = { error: "You must be logged in as admin to create a new project." }
  response.render('login', model)
}
})

```

3) IMAGES UPLOAD: MULTR

Home Projects Skills Contact Hej, admin! Logout

List of Projects

[New project](#)

[Counting people with a camera](#): The purpose of this project is to count people passing through a corridor and to know how many are in the room at a certain time. Year: 2022. [Edit](#) [X](#)

[Visualisation of 3D medical images](#): The project makes a 3D model of the analysis of the body of a person and displays the detected health problems. It is useful for doctors to view in 3D their patients and the evolution of a disease. Year: 2020. [Edit](#) [X](#)

[Multiple questions system](#): During the lockdowns in France, this project was useful to test the students online with a Quizz system. Year: 2021. [Edit](#) [X](#)

[First](#) [1](#) [2](#) [3](#) [Next](#) [Last](#)

Home Projects Skills Contact Hej, admin! Logout

New project

Project

Name:

Description:

Year:

Type:

Image: No file chosen

 Please select a file.

3) IMAGES UPLOAD: MULTR

[Home](#) [Projects](#) [Skills](#) [Contact](#) Hej, admin! [Logout](#)

List of Projects

[New project](#)

[Counting people with a camera](#): The purpose of this project is to count people passing through a corridor and to know how many are in the room at a certain time. Year: 2022. [Edit](#) [X](#)

[Visualisation of 3D medical images](#): The project makes a 3D model of the analysis of the body of a person and displays the detected health problems. It is useful for doctors to view in 3D their patients and the evolution of a disease. Year: 2020. [Edit](#) [X](#)

[Multiple questions system](#): During the lockdowns in France, this project was useful to test the students online with a Quizz system. Year: 2021. [Edit](#) [X](#)

[First](#) [1](#) [2](#) [3](#) [Next](#) [Last](#)

[Home](#) [Projects](#) [Skills](#) [Contact](#) Hej, admin! [Logout](#)

List of Projects

[New project](#)

[Signal 2 Analysis for Detection of Epileptic Diseases](#): This project 3 was about the detection of epileptic problems in signals. Year: 2020. [Edit](#) [X](#)

[My new project](#): I started a cool project this year! Year: 2025. [Edit](#) [X](#)

[First](#) [Previous](#) [1](#) [2](#) [3](#) [Last](#)

3) IMAGES UPLOAD: MULTR

[Home](#) [Projects](#) [Skills](#) [Contact](#) Hej, admin! [Logout](#)

List of Projects

New project

Signal 2 Analysis for Detection of Epileptic Diseases: This project 3 was about the detection of epileptic problems in signals. Year: 2020. Edit X

My new project: I started a cool project this year! Year: 2025. Edit X

First Previous 1 2 3 Last

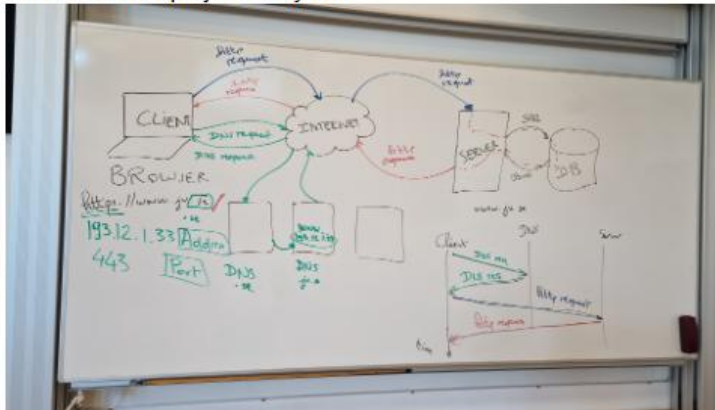
[Home](#) [Projects](#) [Skills](#) [Contact](#) Hej, admin! [Logout](#)

Project details

My new project - 2025

Type: Teaching

I started a cool project this year!



Skills used:
PHP

3) IMAGES UPLOAD: MULTR

[Home](#) [Projects](#) [Skills](#) [Contact](#) Hej, admin! [Logout](#)

List of Projects

[New project](#)

[Counting people with a camera](#): The purpose of this project is to count people passing through a corridor and to know how many are in the room at a certain time. Year: 2022. [Edit](#) [X](#)

[Visualisation of 3D medical images](#): The project makes a 3D model of the analysis of the body of a person and displays the detected health problems. It is useful for doctors to view in 3D their patients and the evolution of a disease. Year: 2020. [Edit](#) [X](#)

[Multiple questions system](#): During the lockdowns in France, this project was useful to test the students online with a Quizz system. Year: 2021. [Edit](#) [X](#)

[First](#) [1](#) [2](#) [3](#) [Next](#) [Last](#)

[Home](#) [Projects](#) [Skills](#) [Contact](#) Hej, admin! [Logout](#)

Modify project

Project

Name:

Description:

Year:

Type:

Image: No file chosen

 Please select a file.

3) IMAGES UPLOAD: MULTER

[Home](#) [Projects](#) [Skills](#) [Contact](#) Hej, admin! [Logout](#)

Project details

Counting people with a camera - 2022

Type: Research

The purpose of this project is to count people passing through a corridor and to know how many are in the room at a certain time.

```
app.post('/login', async (req, res) => {  
  const { username, password } = req.body;  
  if (username === adminUsername && await bcrypt.compare(password, adminPassword)) {  
    req.session.isLoggedIn = true;  
    req.session.username = username;  
    res.redirect('/');  
  } else {  
    res.render('login', { error: 'Invalid username or password' });  
  }  
});
```

Skills used:
Python
Scikit-image
OpenCV

4) USER MANAGEMENT SYSTEM

- For grade 5 (project alone or by pair of two students):
 - You must have a user management system, it means you must have a "**Users**" table with at least an id, a login/username and a password (hashed), the **admin** can do the CRUD on this table: **create, modify/update, read/list and delete** users.
 - The **users** in your table **can log in** on your website.
 - The passwords are **HASHED** (no clear text password!).
- You already did the **CRUD** on another table (requirement for grade 3), it is a **similar code** for the table "Users", plus the hashing of the password using `bcrypt.hash(...)`.

ANY QUESTIONS?